

MSc Computer Science — Distributed Computing

Lab 03

CAP Theorem — AP vs CP Systems

Take-Home Assignment | Go Language | Docker Desktop

Item	Detail
What you build	Two distributed key-value stores: one AP, one CP
Where you run it	Your own laptop — Docker Desktop required
Estimated time	4–5 hours including experiments and discussion questions
Prior lab code	Not required — this assignment is fully independent
Submission	complete the Moodle quiz
Questions	Post to the Moodle Discussion Forum

Background — CAP Theorem

Brewer's CAP theorem states that a distributed system can guarantee at most 2 of these 3 properties simultaneously:

Property	Meaning	Example behaviour during partition
C — Consistency	All nodes return the same data at the same time	Refuses a write rather than risk nodes having different values
A — Availability	Every request always gets a response	Always accepts reads and writes, even if data may be temporarily stale
P — Partition tolerance	System works even if nodes cannot communicate	Non-negotiable in real networks — packets do get lost



The Real Choice

Because partitions WILL happen in any real network, P is not optional. The real design choice is: when a partition occurs, do you stay Consistent (CP) or Available (AP)? CP: refuse writes rather than risk inconsistency. AP: always respond, accept that some reads may be slightly stale.

System	You build	During partition	Real examples
AP Store	Tasks 1–8	Always accepts reads and writes. Nodes sync every	DNS, shopping carts, social media likes

		second. May briefly return stale values.	
CP Store	Tasks 9–15	Requires 3 out of 5 nodes to agree. Refuses writes if majority unreachable. Reads always consistent.	Bank balances, inventory, medical records

Setup — Get Your Environment Running

⚠ Do This First

Complete all setup steps before touching any code. If Docker is not installed and running, nothing else will work.

Step 1 — Install Docker Desktop

Docker Desktop runs containers on your laptop. It is free for students.

Your OS	Download link	Notes
Windows 10/11	https://docs.docker.com/desktop/install/windows-install/	Requires WSL2 — installer will guide you
macOS (Intel)	https://docs.docker.com/desktop/install/mac-install/	Download the Intel chip version
macOS (Apple M1/M2/M3)	https://docs.docker.com/desktop/install/mac-install/	Download the Apple Silicon version
Linux	https://docs.docker.com/desktop/install/linux-install/	Or install Docker Engine + Compose plugin

After installing, open Docker Desktop and wait for it to show the green running status. Then verify in a terminal:

```
docker --version
# Should show: Docker version 24.x.x or higher

docker-compose --version
# Should show: Docker Compose version 2.x.x or higher
# On newer Docker: try 'docker compose version' (no hyphen)
```

Step 2 — Download Lab 03 Files

Download lab03-complete.zip from Moodle and unzip it:

```
# Windows (in PowerShell or WSL terminal):
cd ~/Downloads
unzip lab03-complete.zip
cd lab03-complete

# Mac / Linux:
cd ~/Downloads
unzip lab03-complete.zip
cd lab03-complete

# Verify contents:
ls
# Should show: student/  solution/  docker/  docs/  README.md
```

Step 3 — Build the Docker Image

This compiles the skeleton code and creates a Docker image called lab03. Run from inside the lab03-complete folder:

```
docker build -t lab03 -f docker/Dockerfile .

# Takes 2-3 minutes on first run (downloads Go compiler)
# Expected last line: Successfully tagged lab03:latest

# Verify image was created:
docker images | grep lab03
# Should show: lab03    latest    ...    ~350MB
```

Step 4 — Start All 10 Containers

One command starts 5 AP nodes and 5 CP nodes:

```
docker-compose -f docker/docker-compose.yml up -d

# Verify all 10 are running:
docker ps | grep lab03
# Should show 10 containers:
#   lab03-ap-node1 ... lab03-ap-node5    (AP system on ports 7000-7004)
#   lab03-cp-node1 ... lab03-cp-node5    (CP system on ports 8000-8004)
```

Step 5 — Verify Everything Works

```
# Test AP node 1
docker exec lab03-ap-node1 /lab03/ap/ap_bin -mode cli -port 7000 ping
# Expected: AP node is alive

# Test CP node 1
docker exec lab03-cp-node1 /lab03/cp/cp_bin -mode cli -port 8000 ping
# Expected: CP node is alive
```

✓ If Both Pings Work

Your environment is ready. Move on to Part 1. The pre-compiled binaries run the skeleton code — put and get commands connect but do nothing useful until you implement the TODO functions.

✗ If a Ping Fails

Check Docker Desktop is running (green icon in taskbar/menu bar). Then try: `docker-compose -f docker/docker-compose.yml restart`. If still failing, check: `docker logs lab03-ap-node1` for the error.

Container Reference

Container	System	Port on localhost	Enter with
lab03-ap-node1	AP	7000	<code>docker exec -it lab03-ap-node1 bash</code>
lab03-ap-node2	AP	7001	<code>docker exec -it lab03-ap-node2 bash</code>
lab03-ap-node3	AP	7002	<code>docker exec -it lab03-ap-node3 bash</code>

lab03-ap-node4	AP	7003	docker exec -it lab03-ap-node4 bash
lab03-ap-node5	AP	7004	docker exec -it lab03-ap-node5 bash
lab03-cp-node1	CP	8000	docker exec -it lab03-cp-node1 bash
lab03-cp-node2	CP	8001	docker exec -it lab03-cp-node2 bash
lab03-cp-node3	CP	8002	docker exec -it lab03-cp-node3 bash
lab03-cp-node4	CP	8003	docker exec -it lab03-cp-node4 bash
lab03-cp-node5	CP	8004	docker exec -it lab03-cp-node5 bash

How to Recompile After Editing Code

Every time you edit a source file you must recompile and restart. Do this inside the container:

```
# AP system - run inside lab03-ap-node1
docker exec -it lab03-ap-node1 bash
cd /lab03/ap
pkill ap_bin # stop running process
go build -o ap_bin . # recompile
./ap_bin -mode server -port 7000 \
  -peers ap-node2:7000,ap-node3:7000,ap-node4:7000,ap-node5:7000 &

# CP system - run inside lab03-cp-node1
docker exec -it lab03-cp-node1 bash
cd /lab03/cp
pkill cp_bin
go build -o cp_bin .
./cp_bin -mode server -port 8000 \
  -peers cp-node2:8000,cp-node3:8000,cp-node4:8000,cp-node5:8000 &
```

Important

You must recompile AND restart in EVERY container that runs that code. If you fix node.go in ap-node1, you need to recompile and restart in ap-node2, ap-node3, ap-node4, ap-node5 as well — all 5 containers share the same code.

Useful Commands

```
# Stop all containers
docker-compose -f docker/docker-compose.yml down

# Start all containers again
docker-compose -f docker/docker-compose.yml up -d

# Restart one container (e.g. if it crashed)
docker restart lab03-ap-node3

# View logs of a container
docker logs lab03-ap-node1
docker logs -f lab03-ap-node1 # follow live

# Check all containers are running
docker ps | grep lab03

# CLI shortcut - use from your laptop terminal (not inside container)
docker exec lab03-ap-node1 /lab03/ap/ap_bin -mode cli -port 7000 put city London
docker exec lab03-ap-node5 /lab03/ap/ap_bin -mode cli -port 7000 get city
```


Part 1 — Build the AP System (Tasks 1–8)

Open the AP source files for editing. All files are inside the ap-node1 container at /lab03/ap/

```
docker exec -it lab03-ap-node1 bash
cd /lab03/ap
ls
# node.go  store.go  sync.go  rpc.go  main.go  ap_bin
```

Edit with nano:

```
nano node.go    # Tasks 1-2
nano store.go   # Tasks 3-5
nano sync.go    # Tasks 6-7
nano rpc.go     # Task 8
```

AP Tasks

Task
1
node.g
o

NewNode() [Easy]

Create AP node: set addr, peers, empty store map, syncInterval = 1*time.Second

Task
2
node.g
o

timestamp() [Easy]

Return time.Now().UnixNano() — used as version number for conflict resolution

Task
3
store.g
o

Put() [Easy]

Store key locally with current timestamp — no coordination, always succeeds

Task
4
store.g
o

Get() [Easy]

Return local value immediately — may be stale, that is correct for AP

Task
5
store.g
o

merge() [Medium]

Update store keeping higher timestamp per key — last-write-wins

Task
6
sync.g
o

syncWith() [Medium]

Send local store snapshot to one peer via Sync RPC, merge their response

Task
7
sync.g
o

startSync() [Medium]

Launch goroutine: call syncWith() for every peer on every ticker tick

Task**8**

rpc.go

Put, Get, Sync handlers [Easy]

Each handler calls the correct local function and fills the reply struct

Verify AP System Works

After implementing all 8 tasks and recompiling all 5 AP containers:

```
# Put from node 1
docker exec lab03-ap-node1 /lab03/ap/ap_bin -mode cli -port 7000 put city London
# Expected: Stored key="city" value="London"

# Get from node 5 immediately — may be stale
docker exec lab03-ap-node5 /lab03/ap/ap_bin -mode cli -port 7000 get city
# May return: not found (sync not happened yet — this is correct)

# Wait 3 seconds then get from node 5 again
sleep 3
docker exec lab03-ap-node5 /lab03/ap/ap_bin -mode cli -port 7000 get city
# Expected: key="city" value="London" (synced after 1-3 seconds)
```



Coming Later in the Course

We use `time.Now().UnixNano()` as a version number (Week 10: Time and Synchronisation will cover Vector Clocks — a proper solution). We broadcast to ALL peers every second (Week 6: Message-oriented communication will cover Gossip protocols — a more efficient approach).

Part 2 — Build the CP System (Tasks 9–15)

Open the CP source files:

```
docker exec -it lab03-cp-node1 bash
cd /lab03/cp
nano node.go      # Tasks 9-10
nano store.go     # Tasks 11-12
nano quorum.go    # Tasks 13-14
nano rpc.go       # Task 15
```

CP Tasks

Task

9

node.g
o

`NewNode()` [Easy]

Create CP node: set addr, peers, empty store, quorum = len(peers)/2 + 1

Task

10

node.g
o

`isAlive()` [Easy]

Ping a peer with callRPC — return true if reachable, false if error

Task

11

store.g
o

`Put()` [Hard]

Write to majority — call broadcastWrite, count acks+self, fail if below quorum

Task

12

store.g
o

`Get()` [Hard]

Read from majority — call broadcastRead, add local, return highest timestamp

Task 13

quorum.g
o

`broadcastWrite()` [Medium]

Send Write RPC to all peers concurrently using goroutines and a buffered channel

Task 14

quorum.g
o

`broadcastRead()` [Medium]

Send Read RPC to all peers concurrently, collect Entry responses

Task

15

rpc.go

`Write, Read, Put, Get handlers` [Easy]

Write/Read store locally. Put/Get call quorum functions and set reply.Err on failure

Quorum Explained

With 5 nodes (self + 4 peers), quorum = $5/2 + 1 = 3$. This means:

```
Total nodes = 5 (self + 4 peers)
```

```
Quorum      = 3    (majority)

Put needs 3 acks to succeed:
  self (always counts) + 2 peers = 3    OK
  self (always counts) + 1 peer  = 2    FAIL — quorum not reached

Get needs 3 responses:
  picks the one with the highest timestamp (most recent write)
```

Verify CP System Works

```
# Normal put and get — all 5 nodes up
docker exec lab03-cp-node1 /lab03/cp/cp_bin -mode cli -port 8000 put city London
# Expected: Stored key="city" value="London"

docker exec lab03-cp-node5 /lab03/cp/cp_bin -mode cli -port 8000 get city
# Expected: key="city" value="London" (always consistent!)

# Stop 3 nodes and try to write
docker stop lab03-cp-node3 lab03-cp-node4 lab03-cp-node5
docker exec lab03-cp-node1 /lab03/cp/cp_bin -mode cli -port 8000 put city Paris
# Expected: quorum not reached: got 2 need 3

# Restore nodes
docker start lab03-cp-node3 lab03-cp-node4 lab03-cp-node5
```

Coming Later in the Course

We use simple majority counting here (Week 9: Consensus will cover Raft and Paxos — proper algorithms that survive node crashes, not just partitions). We use `time.Now()` timestamps (Week 10: Time and Synchronisation will cover better approaches).

Part 3 — Experiments: AP vs CP Comparison

⚠ Before Starting Experiments

Both systems must be fully working (all 15 tasks implemented and compiled). Restart all containers first to get a clean state: `docker-compose -f docker/docker-compose.yml down && docker-compose -f docker/docker-compose.yml up -d`

Experiment A — Baseline — Normal Operation

Steps:

1. Verify all 10 containers are running: `docker ps | grep lab03`
2. AP: put 5 different keys from ap-node1, get all 5 from ap-node5 after 3 seconds
3. CP: put 5 different keys from cp-node1, get all 5 from cp-node5

Observe and record:

- > AP: did all 5 gets succeed after 3 seconds? Were values correct?
- > CP: did all 5 gets succeed immediately? Were values correct?

Experiment B — Stale Read — The Core AP vs CP Difference

Steps:

4. AP: put key 'version'='1' from ap-node1
5. Immediately (within 1 second) get 'version' from ap-node5 — record result
6. Wait 3 seconds, get 'version' from ap-node5 again — record result
7. CP: put key 'version'='1' from cp-node1
8. Immediately get 'version' from cp-node5 — record result

Observe and record:

- > AP: what did ap-node5 return immediately? What after 3 seconds?
- > CP: what did cp-node5 return immediately?
- > Which system gave you a stale read and why is this acceptable for AP?

Experiment C — Minority Partition — 2 Nodes Down

Steps:

9. Stop 2 AP nodes: `docker stop lab03-ap-node4 lab03-ap-node5`
10. AP: put key 'status'='active' from ap-node1 — does it succeed?
11. AP: get 'status' from ap-node1, ap-node2, ap-node3
12. Restart: `docker start lab03-ap-node4 lab03-ap-node5`
13. Wait 5 seconds — get 'status' from all 5 AP nodes
14. Repeat steps 1-3 for CP (stop cp-node4, cp-node5)

Observe and record:

- > Did AP accept the write with only 3 nodes? (3 = quorum for 5 nodes)
- > Did CP accept the write with only 3 nodes?
- > After restart, did all AP nodes eventually return the correct value?

Experiment D — Majority Partition — 3 Nodes Down

Steps:

15. Stop 3 AP nodes: `docker stop lab03-ap-node3 lab03-ap-node4 lab03-ap-node5`
16. AP: put key 'alert'='critical' from ap-node1
17. AP: get 'alert' from ap-node1 and ap-node2
18. Restart: `docker start lab03-ap-node3 lab03-ap-node4 lab03-ap-node5`
19. Wait 5 seconds — get 'alert' from all 5 AP nodes
20. Repeat for CP (stop cp-node3, cp-node4, cp-node5)

Observe and record:

- > Did AP write succeed with only 2 nodes available?
- > Did CP write succeed? What exact error message appeared?
- > After AP recovery: are all 5 nodes consistent? How long did it take?

Experiment E — Conflicting Writes — AP Only**Steps:**

21. Block ap-node3 outgoing traffic: `docker exec lab03-ap-node3 iptables -A OUTPUT -j DROP`
22. Write 'score'='100' from ap-node1
23. Write 'score'='999' from ap-node3 (it cannot reach others)
24. Restore: `docker exec lab03-ap-node3 iptables -F`
25. Wait 5 seconds — get 'score' from all 5 AP nodes

Observe and record:

- > What value does each node return for 'score' after recovery?
- > Which value won the conflict? Why? (hint: look at timestamps)
- > Note: CP cannot have conflicting writes by design — skip CP for this experiment

Discussion Questions

Answer all four questions. Submit answers via the Moodle quiz. You may also write answers here for your own reference.

26. In Experiment B, the AP system returned a stale value immediately after a write. How many seconds did it take to converge to the correct value? Which line of code in sync.go controls this? What would happen if you changed it to 10 seconds?

Answer:

27. In Experiment D, the CP system refused writes when 3 nodes were down. The quorum formula is $\text{len(peers)}/2 + 1 = 3$. Why is this threshold 3 and not 2? What problem would occur if you changed the quorum to 2?

Answer:

28. You used `time.Now().UnixNano()` as a version number for conflict resolution in the AP system. Describe a realistic scenario where this approach gives the wrong result. How would vector clocks (covered in Week 10) solve this problem?

Answer:

29. A hospital patient record system vs a Twitter like counter — which should be CP and which AP? Use specific results from your Experiments B and D to justify your answer.

Answer:

Results Recording

Experiment Results

Experiment	AP Result	CP Result
A — All gets succeeded?		
B — Immediate read from node5		
B — Read after 3 seconds		
C — Write with 2 nodes down?		
D — Write with 3 nodes down?		
D — Error message (if any)		N/A
D — Recovery time (seconds)		
E — Winning value after conflict		N/A (CP cannot conflict)

AP vs CP Comparison

Property	AP	CP
Always accepts writes		
Read always returns latest value		
Handles conflicting writes		
Works when majority of nodes down		
Best use case		

Submission

What to Submit

Item	Where	Format
Discussion questions (Q1-Q4)	Moodle — Lab 03 Quiz	Type answers in Moodle
Results tables	Moodle — Lab 03 Quiz	Fill in Moodle form

Submission Checklist

- ☐ AP Tasks 1-8 all implemented and tested
- ☐ AP: put from node1, get from node5 after 3s returns correct value
- ☐ CP Tasks 9-15 all implemented and tested
- ☐ CP: write fails with error when 3 nodes are down
- ☐ Experiments A-E completed for both systems
- ☐ Results table filled in
- ☐ AP vs CP comparison table filled in
- ☐ All 4 discussion questions answered on Moodle

Troubleshooting

Problem	Fix
docker: command not found	Install Docker Desktop from docker.com — make sure it is running
docker-compose: command not found	Try: docker compose (no hyphen) — newer Docker uses this
Cannot build image: go.mod error	Run from inside lab03-complete folder, not a subfolder
Ping fails after building	Run: docker-compose down then docker-compose up -d
ap_bin: no such file	Run: go build -o ap_bin . inside the container
NewNode returned nil	You returned nil in Task 1 — return a &Node{...} pointer
Put does nothing	Check Task 3 stores to n.store — missing write lock?
Sync never converges	Check Task 7 goroutine is actually running — ticker not created?
CP quorum always fails	Check broadcastWrite sends to ackCh for every peer, including failures
iptables permission denied	Run iptables inside the container, not on your laptop
Container keeps restarting	Check logs: docker logs lab03-ap-node1
Port already in use	Run: docker-compose down, then up again

Go Quick Reference

Timestamps and Tickers

```
import "time"

// Version number
ts := time.Now().UnixNano() // int64 nanoseconds

// Periodic background task
go func() {
    ticker := time.NewTicker(1 * time.Second)
    for range ticker.C {
        // runs every second
    }
}()
```

Goroutines + Channels for Quorum

```
// Send RPC to N peers concurrently, collect results
resultCh := make(chan bool, len(n.peers)) // buffered!

for _, peer := range n.peers {
    go func(p string) { // pass peer as argument - avoid loop variable bug
        err := callRPC(p, "Method", args, reply)
        resultCh <- (err == nil)
    }(peer)
}

// Collect ALL results (must read from channel for every goroutine launched)
count := 0
for i := 0; i < len(n.peers); i++ {
    if <-resultCh { count++ }
}
```

Mutex Patterns

```
// Write lock - exclusive access
n.mu.Lock()
n.store[key] = entry
n.mu.Unlock()

// Read lock - allows concurrent readers
n.mu.RLock()
val, ok := n.store[key]
n.mu.RUnlock()

// NEVER call RPC while holding a lock - can deadlock
// Take snapshot, release lock, then call RPC
n.mu.RLock()
snapshot := make(map[string]Entry)
for k, v := range n.store { snapshot[k] = v }
n.mu.RUnlock()
callRPC(peer, "Method", &SyncArgs{Store: snapshot}, &reply)
```